PREDICTIVE MODELING OF CUSTOMER PROMOTION ACCEPTANCE IN SME, LOGISTIC REGRESSION APPROACH

Prepared by:
Floriand BESHKU

Connecticut
USA

2026

PAGE INDEX

Project Summary 3

Business Context & Problem Definition 4

Data Description & Preparation 5

Methodology 8

Analysis & Model Results 11

Reduced Model 15

Interactive Promo Acceptance Calculator 17

Business Implications & Recommendation 18

Appendix (Python Codes) 19

PROJECT SUMMARY

This project demonstrates how data can be used to improve promotional decision-making by replacing intuition-based marketing with an objective, data-driven approach. Rather than sending promotions to all customers or relying on assumptions, the goal of this analysis is to predict which customers are most likely to accept promotion (for example a subscription) and to explain which customer characteristics and behaviors meaningfully influence that decision.

For example, Managers in a gym business often assume that customers who live closer to the facility or who recently engage with marketing messages are more likely to respond positively to promotions. While these intuitions are reasonable, they are often not supported by a structured method to quantify how much each factor actually matters. This project addresses that gap by applying data science techniques to estimate the probability that a customer will accept a promotion based on observable attributes, including demographic information, distance from the gym, digital engagement (email opens and SMS clicks), prior membership history, trial class participation, and promotion characteristics such as discount size and type. Instead of relying on broad assumptions like “engaged customers are more likely to join,” the model produces a clear probability score that reflects how each factor contributes to the likelihood of acceptance.

The analysis is based on a dataset of 5,000 gym customers, where each row represents an individual and multiple variables summarize customer background, engagement behavior, and exposure to promotional offers. Before modeling, the data was reviewed, including the creation of indicator variables for categorical features and the partitioning of the data into training (to train the model), validation (to validate the mode), and test (to test the model) sets. A logistic regression model was then developed and evaluated using confusion matrices, classification metrics, and probability threshold analysis. If any of this sounds unfamiliar at first, there is no need to worry, this report is intentionally written in a simple, practical way so it remains understandable for both technical and non-technical readers. The results show that promotion acceptance can be predicted consistently, and that adjusting probability thresholds allows the business to balance outreach coverage and targeting precision based on strategic goals.

Ultimately, this project delivers both predictive insight and practical business value. The model build enables the business to rank customers by their likelihood of accepting a promotion and to tailor targeting strategies to objectives such as minimizing marketing costs or maximizing new memberships. Furthermore, a reduced version of the model achieves comparable performance while relying on fewer variables, making it easier to explain, maintain, and deploy in real-world systems such as CRM or marketing automation platforms, while also limiting unnecessary data usage. Although the analysis is presented in a gym business context, the same predictive framework is broadly applicable. Any business that runs promotional campaigns, such as fitness centers, retail stores, subscription services, or service providers, can use same approach to improve targeting efficiency, reduce wasted outreach, and support data-driven decisions.

BUSINESS CONTEXT & PROBLEM DEFINITION

In many small and medium-sized businesses, promotional decisions are still driven largely by intuition, prior experience, or broad assumptions about customer behavior. While this approach may be sufficient at a small scale, it often leads to inefficient use of marketing resources as businesses grow, expand their customer base, and increase investment in digital outreach. Promotions are frequently sent to large segments of customers without clear evidence of who is most likely to respond, resulting in wasted marketing spend and missed opportunities to convert high-potential customers.

This challenge is especially relevant in subscription-based businesses where customer acquisition and retention depend heavily on individual characteristics and engagement history. For example, gyms routinely offer incentives such as discounted memberships, free trials, or waived enrollment fees, yet these offers are often distributed broadly rather than strategically. As a result, many customers who receive promotions have little intention of joining, while others who are more receptive may not be prioritized. Without a systematic approach, it is difficult for management to understand which promotional strategies are effective and why.

The core business problem addressed in this project is the lack of a reliable, data-driven method for targeting promotions. Management needs a way to identify which customers are most likely to accept a promotional offer before allocating marketing resources. Doing so would allow the business to reduce unnecessary outreach, improve conversion rates, and justify promotional decisions with measurable evidence rather than assumptions.

The central question guiding this analysis is therefore: **Can customer-level data be used to reliably predict promotion acceptance, and which customer characteristics and behaviors most strongly influence that likelihood?** By answering this question, the project aims to provide a practical decision-support framework that helps the business target promotions more effectively, improve return on marketing investment, and support scalable, data-driven growth.

DATA DESCRIPTION & PREPARATION

The analysis is based on a customer-level dataset of 5,000 gym customers, where each record represents an individual who has been exposed to or is eligible for a gym promotional offer. The dataset reflects customer behavior in a gym and fitness business, capturing differences in demographics, engagement patterns, prior membership history, and responsiveness to marketing efforts. The goal of the dataset is to support predictive modeling of promotion acceptance in a practical business setting. The dataset includes three main categories of information:

A) Customer identifiers and reference information

- customer_id: Unique identifier for each customer (used only for tracking and reference, not as a model input).

B) Customer characteristics and behavioral variables

These variables describe customer background, proximity to the gym, historical engagement, and usage patterns:

- age: Customer age
- gender: Customer gender
- income_k: Annual income in thousands of dollars
- student: Indicator of student status
- distance_miles: Distance between the customer's residence and the gym
- commute_minutes: Estimated travel time to the gym in minutes
- site_visits_30d: Number of visits to the gym's website in the last 30 days
- emails_opened_30d: Number of marketing emails opened in the last 30 days
- sms_clicked_30d: Number of SMS marketing messages clicked in the last 30 days
- app_installed: Indicator of whether the customer has installed the gym's mobile app
- trial_class_attended: Indicator of whether the customer attended a trial class

C) Membership history, engagement depth, and preference variables

This group of variables reflects customer experience with the gym, historical activity, and personal preferences:

- tours_completed: Indicator of whether the customer completed a gym tour
- past_classes_90d: Number of classes attended in the past 90 days
- time_since_last_visit_days: Number of days since the customer's most recent visit
- prior_member: Indicator of whether the customer was previously a gym member
- membership_status: Current membership status (e.g., Never Member, Cancelled)
- price_sensitivity: Customer sensitivity to price (Low, Medium, High)
- fitness_goal: Primary fitness objective (e.g., Weight Loss, General Health)
- preferred_time: Preferred workout time (Morning, Afternoon, Evening)

D) Promotion-related and outcome variables

The dataset also includes information describing the promotional offer and the customer's response:

- lead_source: Origin of the customer lead (e.g., Referral,Walk-in,GoogleAds,Employer Partner)
- contact_channel: Primary communication channel used (Email, SMS, Phone)
- promo_type: Type of promotion offered (Free Trial, Percent Off, Free Enrollment)
- discount_pct: Percentage discount associated with the promotion
- satisfaction_score: Self-reported or inferred satisfaction score
- monthly_spend_usd: Estimated monthly spending on gym-related services

E) Target Variable

- **accept_promo**: Binary outcome variable indicating whether the customer accepted the gym promotion. 1 = Accepted and 0 = Did not accept

Data Preparation

Before modeling, the data was carefully reviewed and prepared to ensure consistency and suitability for the logistic regression analysis. The customer identifier was removed, and all categorical variables were converted into binary indicator (dummy) variables. Continuous variables were retained in their numeric form to preserve interpretability. The dataset was then partitioned into training, validation, and test sets, allowing for model development, threshold tuning, and unbiased performance evaluation. This preparation ensures that the resulting predictions are both reliable and interpretable for real-world promotional decision-making.

Header of the dataset:

customer_id	age	gender	income_k	distance_miles	commute_minutes	student	lead_source
C00001	57	Female	39.5	0.5	4	No	Referral
C00002	43	Female	108.8	1.3	10	Yes	Walk-in
C00003	33	Female	96.8	8.1	46	No	Employer Partner
C00004	25	Female	59.3	1.7	9	No	Google Ads
C00005	56	Male	98.9	6.5	49	No	Referral

contact_channel	site_visits_30d	emails_opened_30d	sms_clicked_30d	app_installed	trial_class_attended
SMS	4	2	1	Yes	No
Phone	8	2	1	Yes	No
SMS	6	1	2	Yes	Yes
Email	4	1	2	No	No
SMS	4	7	0	Yes	No

tours_completed	past_classes_90d	time_since_last_visit_days	prior_member	membership_status
No	0	37	Yes	Cancelled
No	2	28	No	Never Member
No	3	159	No	Never Member
Yes	2	13	No	Never Member
No	10	74	No	Never Member

price_sensitivity	fitness_goal	preferred_time	promo_type	discount_pct	satisfaction_score
Low	General Health	Afternoon	Free Enrollment	8	9.1
Low	Weight Loss	Evening	Free Trial	12	7.4
Med	Weight Loss	Evening	Free Trial	12	7
Med	General Health	Morning	Percent Off	10	7.6
High	General Health	Evening	Free Enrollment	8	7.1

monthly_spend_usd	accept_promo
59.93	0
8.25	0
44.5	0
1.9	0
1.06	0

METHODOLOGY

As mentioned before, the goal of this analysis is to predict whether a customer will accept a gym subscription promotion based on observable characteristics and engagement behavior. The approach uses a statistical model to estimate the probability of acceptance for each customer. These probability estimates allow the business to target promotions more effectively and make informed decisions about how aggressively to market offers.

1) What information is used to predict promotion acceptance

The model relies only on measurable customer attributes and behaviors that are relevant to promotional responsiveness. These include:

Customer background and accessibility

- Age, income level, student status
- Distance from the gym and estimated commute time

Engagement and interaction behavior

- Website visits in the last 30 days
- Emails opened and SMS messages clicked
- Mobile app installation
- Trial class attendance and completed gym tours

Membership history and activity

- Prior membership status
- Current membership status (e.g., never member, cancelled)
- Classes attended in the past 90 days
- Time since last gym visit

Promotion and marketing attributes

- Lead source (referral, walk-in, advertising, employer partner)
- Contact channel (email, SMS, phone)
- Promotion type and discount size
- Price sensitivity and satisfaction score

These variables collectively capture customer motivation, engagement depth, proximity, and promotional appeal, which are central drivers of promotion acceptance in a gym business.

2) Preparing the data for fair and reliable modeling

Because the dataset includes a mix of numeric and categorical variables, several preparation steps were required before modeling:

- Customer identifiers were removed to avoid bias.
- Some categorical variables (such as lead source, contact channel, and promotion type) were converted into binary indicator variables.

- Numeric variables were retained in their original scale to preserve interpretability in the regression model.
- The dataset was split into training, validation, and test sets to ensure that model performance could be evaluated on unseen data.

These steps ensure that the model learns meaningful patterns rather than artifacts of data structure or formatting.

3) Predictive modeling approach: Logistic Regression

Logistic regression was selected as the primary modeling technique because it is well suited for binary outcomes and produces results that are interpretable and actionable. Instead of predicting a simple yes/no outcome, logistic regression estimates the **probability** that a customer will accept the promotion.

This probability-based output is especially valuable for business decision making because it allows customers to be ranked by likelihood of acceptance rather than treated equally. The model learns how each variable increases or decreases the odds of accepting a promotion, making it possible to understand not only *who* is likely to accept, but also *why*.

4) Model evaluation and threshold selection

Model performance was evaluated using confusion matrices and classification metrics on the training, validation, and test sets. In addition to the default probability threshold of 0.50, a lower business-oriented threshold of 0.30 was also examined.

Changing the threshold, we adjust how aggressive the promotional targeting strategy is. A lower threshold increases the number of customers flagged as likely accepters, improving reach and customer acquisition at the cost of sending some promotions to customers who may not convert. This threshold analysis allows the business to align the model's use with strategic goals such as growth or cost control.

5) Variable selection and model simplification

To improve interpretability and focus on the most impactful predictors, a reduced logistic regression model was also developed based on statistical significance (p-values). Variables that did not meaningfully contribute to explaining promotion acceptance were removed, resulting in a more concise model with similar or improved performance. This step ensures that the final model highlights the customer behaviors and characteristics that truly matter for promotion acceptance, making the results easier to communicate to non-technical stakeholders and more useful for decision-making.

ANALYSIS & RESULTS

Model Overview and Data Split

The logistic regression model was trained and evaluated using a train/validation/test split of 50% / 30% / 20%, ensuring that model performance could be assessed on data not used during training. Out of the 5,000 total customers, 2,500 were used for model training, 1,500 for validation, and 1,000 for final testing.

To predict whether a customer will accept a gym promotion, a logistic regression model was developed using customer demographics, engagement behavior, prior membership history, and promotion characteristics. This probability can then be converted into a decision using a chosen threshold that reflects business priorities.

The fitted model estimates the following relationship:

$$p(\text{accept}) = \frac{1}{1 + \exp\left(-\left(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k\right)\right)}$$

where:

- p is the probability that a customer accepts the promotion,
- $\beta_0$is the intercept,
- $\beta_k$are the coefficients associated with each customer characteristic x_k.

A positive coefficient increases the likelihood of accepting the promotion, while a negative coefficient decreases it, holding all other variables constant.

Logistic Regression Coefficients (including Intercept):

	Feature	Coefficient
0	Intercept	-3.582312
1	age	0.003036
2	income_k	0.009359
3	distance_miles	0.004675
4	commute_minutes	-0.022529
5	site_visits_30d	-0.018427
6	emails_opened_30d	0.093035
7	sms_clicked_30d	0.164597
8	past_classes_90d	0.091878
9	time_since_last_visit_days	-0.023522
10	discount_pct	0.027167
11	satisfaction_score	0.111062
12	monthly_spend_usd	-0.000755
13	gender_Male	-0.260534
14	student_Yes	0.006432
15	lead_source_Google Ads	0.081593
16	lead_source_Instagram	0.001818
17	lead_source_Other	0.187346
18	lead_source_Referral	0.619180
19	lead_source_Walk-in	0.397699
20	contact_channel_Phone	-0.346110
21	contact_channel_SMS	0.190314
22	app_installed_Yes	0.095240
23	trial_class_attended_Yes	0.931905
24	tours_completed_Yes	0.760164
25	prior_member_Yes	0.227027
26	membership_status_Cancelled	0.569537
27	membership_status_Frozen	-0.432610
28	membership_status_Never Member	-0.211225
29	price_sensitivity_Low	-0.099749
30	price_sensitivity_Med	-0.384031
31	fitness_goal_Rehab	-0.043077
32	fitness_goal_Strength	-0.037560
33	fitness_goal_Weight Loss	-0.195149
34	preferred_time_Evening	0.169485
35	preferred_time_Morning	0.009926
36	promo_type_Free Trial	0.171340
37	promo_type_Percent Off	0.195515
38	competitor_nearby_Yes	-0.266848

On the left you can see our regression model coefficients, computed on python programing language. Each coefficient represents the direction and strength of the relationship between a customer characteristic and the likelihood of accepting the promotion, holding all other variables constant. A positive coefficient means that as the variable increases (or when the condition is true), the probability of accepting the promotion increases. A negative coefficient means that the variable reduces the likelihood of accepting the promotion. The magnitude of the coefficient indicates how influential that variable is relative to others. The intercept represents the baseline tendency to accept the promotion when all other variables are zero (or at their reference category). By itself, it is not directly interpretable, but it anchors the model's probability calculations.

Examples of Key Coefficients:

1. Trial Class Attended (+0.93)

This is one of the strongest positive coefficients in the model. Customers who attended a trial class are much more likely to accept the promotion compared to customers who did not.

2. SMS Clicked in the Last 30 Days (+0.16)

This coefficient captures recent engagement with marketing messages. Customers who actively click on SMS messages are more responsive and therefore more likely to accept a promotion.

3. Lead Source Referral (+0.62)

This variable compares referral customers to the baseline lead source. Customers who come through referrals are substantially more likely to accept promotions.

The Logistic Regression Formula

The Logistic Regression Formula represents the mathematical logic the model uses to estimate the probability that a given customer will accept a gym promotion. Each term in the formula adjusts the likelihood of acceptance either upward or downward depending on how that factor historically influences customer behavior.

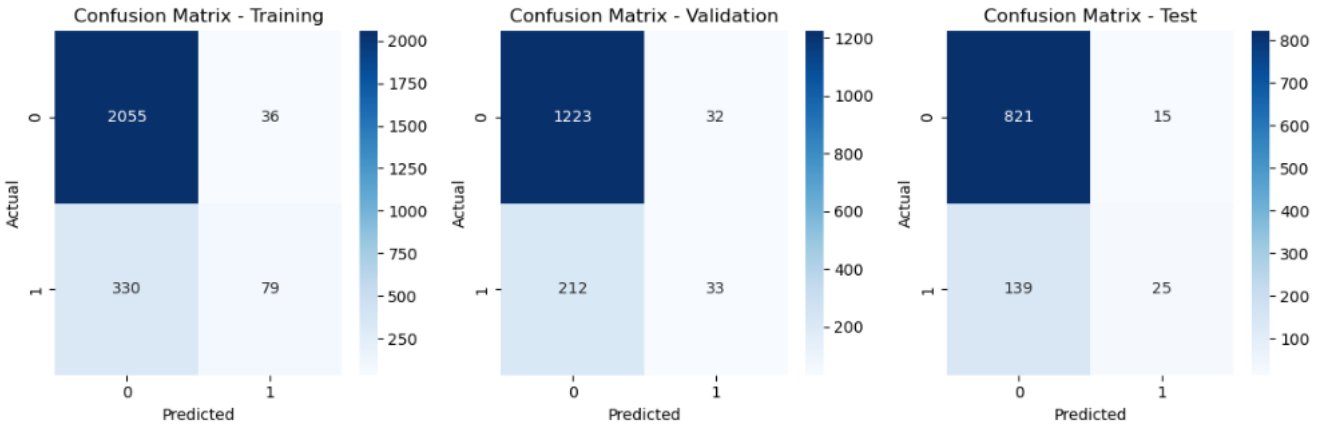
```
Logistic Regression Formula (probability of accept_promo = 1):
p = 1 / (1 + exp(-(-3.5823 + 0.0030*age + 0.0094*income_k + 0.0047*distance_miles + -0.0225*commute_minutes + -0.0184*site_visits_30d + 0.0930*emails_ope
ned_30d + 0.1646*sms_clicked_30d + 0.0919*past_classes_90d + -0.0235*time_since_last_visit_days + 0.0272*discount_pct + 0.1111*satisfaction_score + -0.00
08*monthly_spend_usd + -0.2605*gender_Male + 0.0064*student_Yes + 0.0816*lead_source_Google Ads + 0.0018*lead_source_Instagram + 0.1873*lead_source_Other
+ 0.6192*lead_source_Referral + 0.3977*lead_source_Walk-in + -0.3461*contact_channel_Phone + 0.1903*contact_channel_SMS + 0.0952*app_installed_Yes + 0.93
19*trial_class_attended_Yes + 0.7602*tours_completed_Yes + 0.2270*prior_member_Yes + 0.5695*membership_status_Cancelled + -0.4326*membership_status_Froze
n + -0.2112*membership_status_Never Member + -0.0997*price_sensitivity_Low + -0.3840*price_sensitivity_Med + -0.0431*fitness_goal_Rehab + -0.0376*fitness
_goal_Strength + -0.1951*fitness_goal_Weight Loss + 0.1695*preferred_time_Evening + 0.0099*preferred_time_Morning + 0.1713*promo_type_Free Trial + 0.1955
*promo_type_Percent Off + -0.2668*competitor_nearby_Yes)))
```

The result of the formula is a probability value between 0 and 1 (0% to 100%), which indicates how likely a specific customer is to accept the promotion.

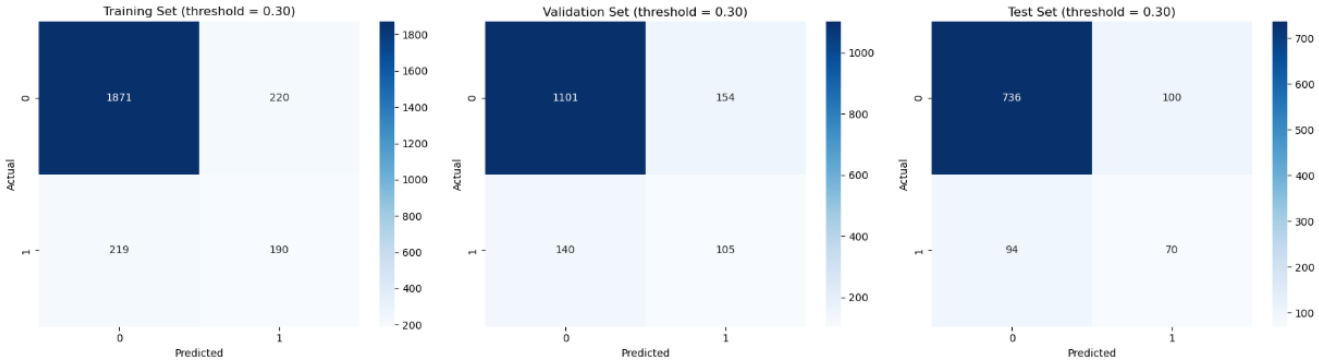
This probability does not make a decision by itself; instead, it allows us to apply a chosen threshold (In our case 50% and 30%) to decide which customers should be targeted. In practice, the formula enables the gym to move from intuition-based marketing to data-driven decision-making by identifying customers who are statistically more likely to respond positively to promotional offers.

For example, if the formula produces a probability of 45% for a particular customer, this means the model estimates that the customer has a 45% chance of accepting the promotion based on their profile and past behavior. On its own, this value is simply a likelihood score. Whether the customer is actually targeted depends on the business threshold chosen. If the gym uses a 50% threshold, this customer would not be selected because their likelihood is below the cutoff. However, if the gym uses a more aggressive 30% threshold, the same customer would be included in the campaign. This illustrates how the model supports flexible marketing decisions: the probability is calculated, but the business can adjust its targeting strategy depending on goals such as maximizing conversions, expanding reach, or controlling promotion costs.

When the threshold is set to 50%, the model targets customers when they are very confident, they will accept the promotion. This results in a very conservative strategy: most customers who are contacted truly are not interested, but many customers who would have accepted the promotion are missed. In the test results, only a small portion of actual promotion acceptances were identified, meaning the gym would lose many potential sign-ups. This approach reduces unnecessary outreach but limits growth and promotion reach. **Threshold 50%:**



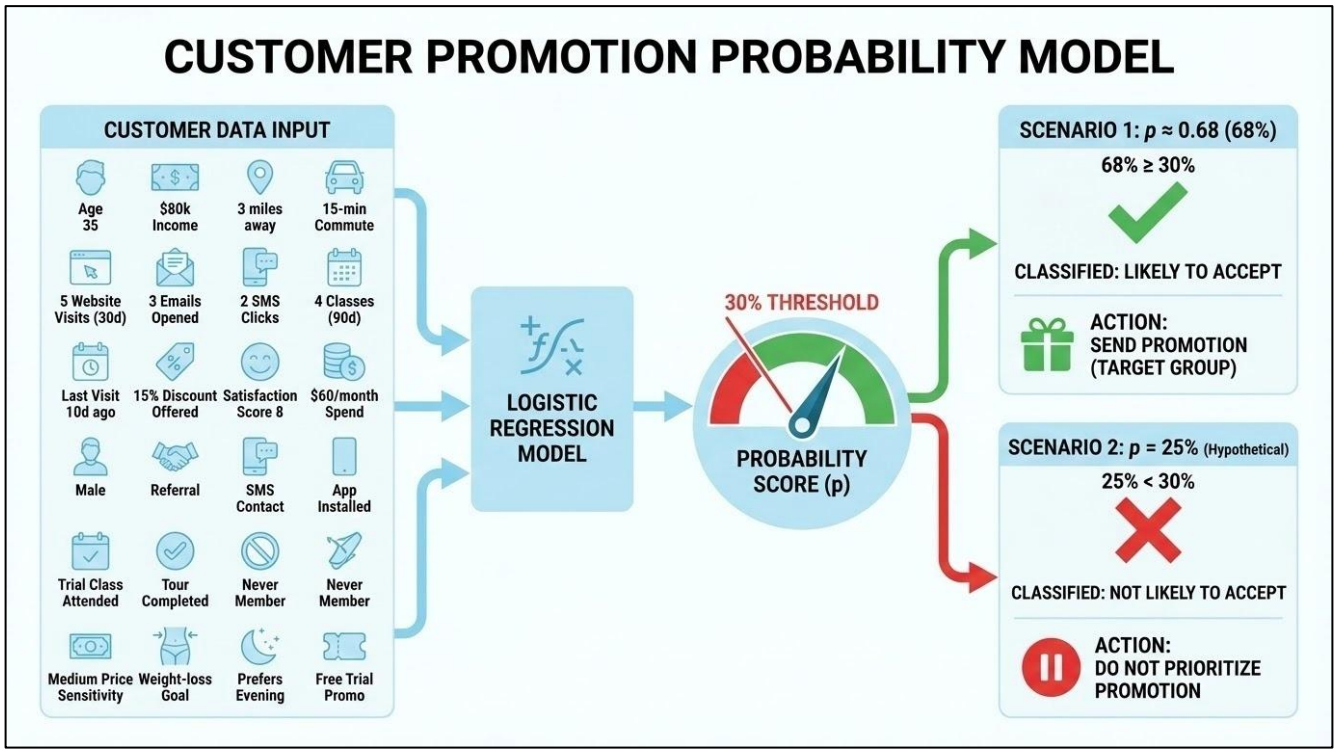
When the threshold is lowered to 30%, the model becomes more flexible and targets customers who show moderate interest in the promotion. This allows the gym to identify significantly more customers who end up accepting the offer. Although more customers who do not accept the promotion are also contacted, the overall number of successful sign-ups increases substantially. Because marketing outreach such as email or SMS has a relatively low cost, this trade-off is often worthwhile. **Threshold 30%:**



In summary, a 50% threshold favors caution and efficiency, while a 30% threshold favors opportunity and growth. The model itself does not change; only the business rule for deciding whom to target changes. This gives the gym the ability to adjust its marketing strategy based on how aggressively it wants to pursue new members, using data-driven probabilities rather than intuition.

How to apply in our case the model

Suppose we evaluate one customer with these characteristics: **Age 35**, **income = $80k**, **lives 3 miles away**, **15-minute commute**, **5 website visits in the last 30 days**, **opened 3 emails**, **clicked 2 SMS messages**, **took 4 classes in the last 90 days**, **last visited 10 days ago**, offered a **15% discount**, has **satisfaction score = 8**, spends **$60/month**, is **male**, came from a **referral**, is contacted by **SMS**, has the **app installed**, **attended a trial class**, **completed a tour**, is a **never member**, has **medium price sensitivity**, has a **weight-loss goal**, prefers **evening**, and is offered a **Free Trial** promo. When we plug these values into the logistic regression formula (in page 12), the model produces a score that converts to a probability of about **p ≈ 0.68 (68%)**. Using the **30% threshold**, this customer is classified as **"likely to accept"** because **68% ≥ 30%**, so **yes, we would send the promotion** (they are in the target group). If the customer's probability had instead been, for example, **p = 0.25 (25%)**, then **25% < 30%**, and the model would classify them as **"not likely to accept"**, meaning **we would not prioritize sending the promotion** under the 30% rule.



REDUCED MODEL

After estimating the full logistic regression model, a variable selection step was performed to simplify the model and improve interpretability, based on statistical significance (p-value). P-values indicate how strongly each variable is associated with promotion acceptance, after accounting for all other variables in the model.

All the variables were ranked by p-value, from most statistically significant to least significant. Using a standard threshold of $p < 0.05$, we retained only the predictors that show a statistically reliable relationship with promotion acceptance. This approach helps remove noise, reduces over-complexity, and ensures that the final model focuses on the most meaningful drivers of customer behavior. The analysis identified 15 statistically significant predictors which are used in our final model.

Using only these 15 significant predictors, a reduced logistic regression model was re-trained. The reduced model preserves predictive performance while being substantially easier to explain and operate. From a business perspective, this simplification makes the model more actionable: marketing and sales teams can focus on a smaller set of high-impact signals when deciding which customers to target, rather than tracking dozens of weak or redundant variables.

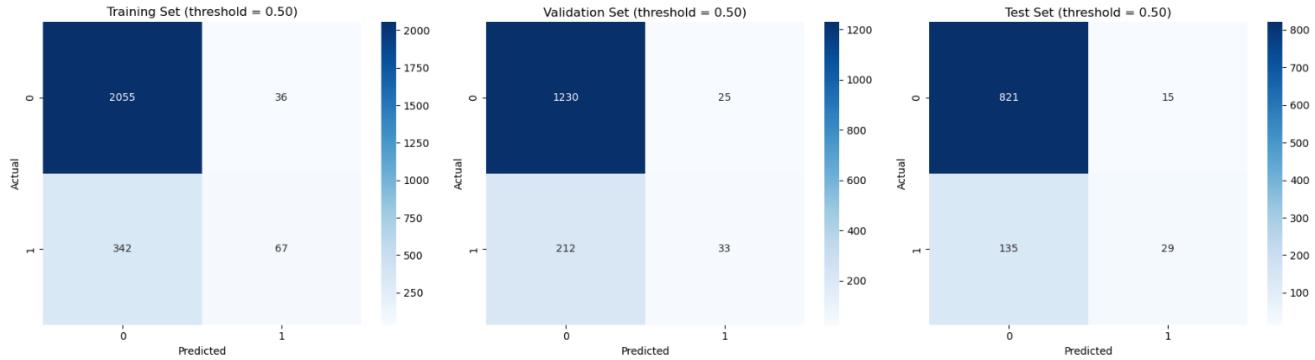
The new reduced logistic regression formula:

```
Reduced Logistic Regression Formula (probability of accept_promo = 1):
p = 1 / (1 + exp(-(-3.4933 - 0.0235*time_since_last_visit_days + 0.9434*trial_class_attended_Yes + 0.7414*tours_completed_Yes + 0.0094*income_k + 0.0947*
past_classes_90d + 0.1549*sms_clicked_30d + 0.5095*lead_source_Referral - 0.2907*price_sensitivity_Med + 0.0329*discount_pct + 0.9160*membership_status_C
ancelled + 0.1125*satisfaction_score - 0.0216*commute_minutes + 0.0854*emails_opened_30d - 0.2422*gender_Male - 0.2552*competitor_nearby_Yes)))
```

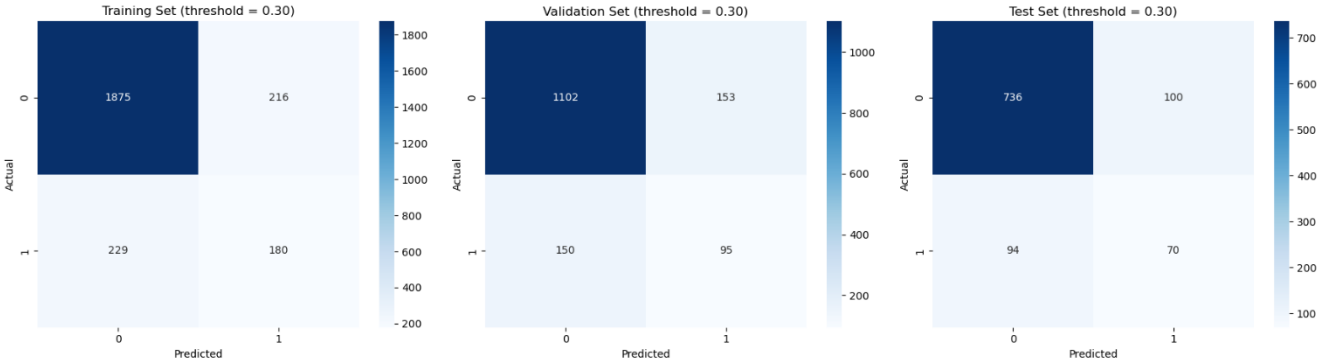
Suppose we evaluate one customer with these characteristics (using the reduced model variables): income_k = 80 ($80k), time_since_last_visit_days = 10, past_classes_90d = 4, emails_opened_30d = 3, sms_clicked_30d = 2, commute_minutes = 15, discount_pct = 15, satisfaction_score = 8, and the "Yes/No" indicators are: trial_class_attended_Yes = 1, tours_completed_Yes = 1, lead_source_Referral = 1, gender_Male = 1, price_sensitivity_Med = 1, competitor_nearby_Yes = 0, membership_status_Cancelled = 0. When we plug these values into the reduced logistic regression formula, the model returns an estimated probability of about $p \approx$ **0.509 (51%).** Using the 30% threshold, this customer is classified as "**likely to accept**" because 51% ≥ 30%, so **yes**; this customer should be included in the promotion target list.

Now comparing that to a different customer where the model produces a lower probability. For example: income_k = 70 ($70k), time_since_last_visit_days = 15, past_classes_90d = 2, emails_opened_30d = 3, sms_clicked_30d = 2, commute_minutes = 20, discount_pct = 15, satisfaction_score = 8, with indicators trial_class_attended_Yes = 0, tours_completed_Yes = 1, lead_source_Referral = 1, gender_Male = 1, price_sensitivity_Med = 1, competitor_nearby_Yes = 1, membership_status_Cancelled = 0. Plugging these into the formula gives $p \approx$ **0.27 (27%)**. With the 30% threshold, this customer is classified as "not likely to accept" because 27% < 30%, so we would not prioritize sending the promotion to this customer under the 30% rule.

With a 50% threshold, the model is strict: it only labels a customer as “likely to accept” when the predicted probability is at least 50%. On the test set, it correctly identified 29 customers who accepted the promo (true positives), but it missed 135 customers who actually accepted (false negatives). It also incorrectly targeted only 15 customers who did not accept (false positives). In business terms: fewer wasted promotions, but many missed opportunities (it fails to capture most customers who would accept).



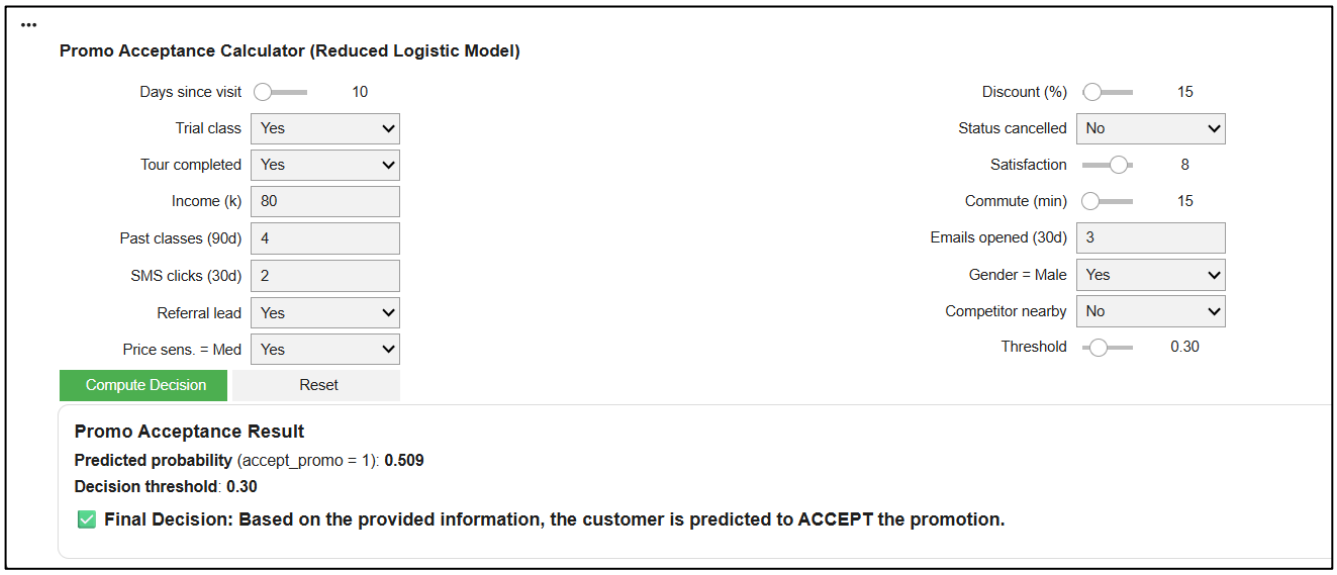
With a 30% threshold, it targets customers when predicted probability is at least 30%. On the test set, it correctly identified 70 customers who accepted (true positives), and it missed 94 acceptors (false negatives). However, it also targeted 100 customers who did not accept (false positives). In business terms: it captures far more potential acceptors, but it also sends promotions to more people who will not accept. This is the classic trade-off: lower threshold = higher reach and higher conversions, but more marketing cost/waste.



INTERACTIVE PROMO ACCEPTANCE CALCULATOR

The figure presents an interactive Promo Acceptance Calculator built using our reduced logistic regression model. The tool allows to manually input at a customer-level and based on these inputs, the application computes the predicted probability that a customer will accept the promotional offer.

So, the model development and statistical analysis performed for the business case are converted into a user-friendly interface that allows users to interact with the model without direct access to the underlying code.



BUSINESS IMPLICATIONS & RECOMMENDATIONS

This project demonstrates how the gym can move from broad, intuition-based promotions to a targeted, data-driven marketing strategy. Instead of sending promotions to all customers equally, the logistic regression model estimates the probability that each individual customer will accept a promotional offer, allowing the business to prioritize outreach where it is most likely to succeed.

The results show that customer engagement and prior interaction with the gym are the strongest drivers of promotion acceptance. Customers who attended a trial class, completed a gym tour, or participated in classes recently are significantly more likely to accept an offer. Similarly, customers who clicked SMS messages, opened emails, or were referred by existing members show higher responsiveness. On the other hand, long gaps since the last visit, longer commute times, medium price sensitivity, and the presence of nearby competitors reduce the likelihood of acceptance. This insight allows the gym to focus promotional efforts on customers who are already warm leads rather than cold prospects.

From a practical business perspective, the model can be used in two different ways depending on how aggressive the gym wants to be with its marketing budget. Using a 50% threshold is a conservative strategy. Under this rule, promotions are sent only to customers who are very likely to accept. This minimizes marketing waste and protects margins, but it captures only a small portion of potential responders. This approach is appropriate when promotions are costly or capacity is limited.

Using a 30% threshold represents a more growth-oriented strategy. While this approach includes some customers who may not ultimately accept, it significantly increases the number of true acceptances captured by the campaign. For a promotion, where the marginal cost of sending an email or SMS is low and the upside of gaining a new or returning member is high, the 30% threshold is more aligned with business objectives. It allows the business to reach customers who are moderately likely to convert and would otherwise be missed under a stricter rule.

Importantly, the reduced model achieves performance comparable to the full model while relying on a smaller set of variables. This makes the model easier to maintain, explain, and deploy within real business systems such as CRM platforms or marketing automation tools. By using fewer data points and focusing only on the most relevant signals, the approach also better respects customer privacy, avoiding unnecessary use of personal information. As a result, marketing and operations teams can confidently apply the model's insights in daily decision-making without needing advanced technical expertise, while aligning with responsible and privacy-aware data practices.

In summary, the recommended approach is to use the reduced logistic regression model with a 30% probability threshold as the primary targeting rule. Over time, the business (the gym in our case) can further retrain the model on new data and refine thresholds based on capacity, seasonality, and campaign performance.

Appendix (Technical & Codes used for this analysis)

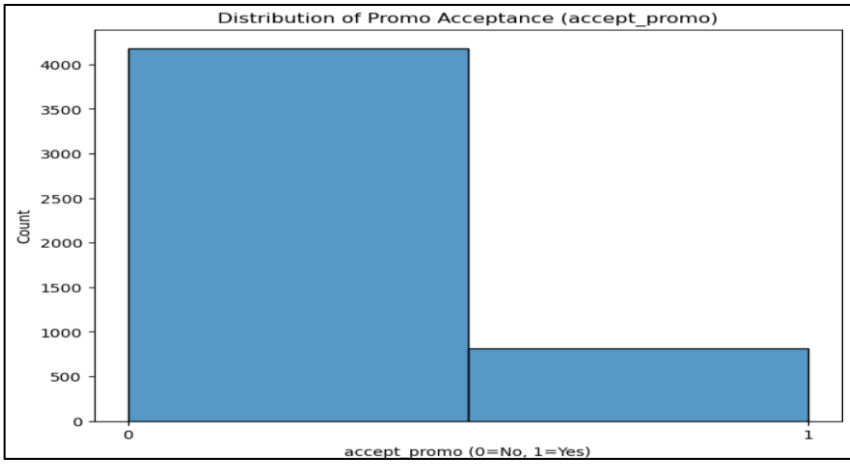
```python
# Load the data & basic inspection
df = pd.read_csv("gym_promo_customers_5000_improved.csv")
# Shape
print("Dataset shape:", df.shape)
# Preview first rows
print("\nFirst 5 rows:")
print(df.head())
# Data types and missing values
print("\nData info:")
print(df.info())
```

```
Dataset shape: (5000, 28)

First 5 rows:
  customer_id  age  gender  income_k  distance_miles  commute_minutes student  \
0      C00001   57  Female      39.5             0.5                4      No
1      C00002   43  Female     108.8             1.3               10     Yes
2      C00003   33  Female      96.8             8.1               46      No
3      C00004   25  Female      59.3             1.7                9      No
4      C00005   56    Male      98.9             6.5               49      No

        lead_source contact_channel  site_visits_30d  ...  membership_status  \
0          Referral             SMS                4  ...          Cancelled
1           Walk-in           Phone                8  ...       Never Member
2  Employer Partner             SMS                6  ...       Never Member
3        Google Ads           Email                4  ...       Never Member
4          Referral             SMS                4  ...       Never Member

   price_sensitivity    fitness_goal preferred_time       promo_type  \
0                Low  General Health      Afternoon  Free Enrollment
1                Low     Weight Loss        Evening       Free Trial
2                Med     Weight Loss        Evening       Free Trial
3                Med  General Health        Morning      Percent Off
4               High  General Health        Evening  Free Enrollment

   discount_pct  satisfaction_score competitor_nearby monthly_spend_usd  \
0             8                 9.1               No             59.93
1            12                 7.4               No              8.25
2            12                 7.0              Yes             44.50
3            10                 7.6               No              1.90
4             8                 7.1               No              1.06
```

```
  accept_promo
0            0
1            0
2            0
3            0
4            0

[5 rows x 28 columns]

Data info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 28 columns):
```

```
Data info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 28 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   customer_id                 5000 non-null   object
 1   age                         5000 non-null   int64
 2   gender                      5000 non-null   object
 3   income_k                    5000 non-null   float64
 4   distance_miles              5000 non-null   float64
 5   commute_minutes             5000 non-null   int64
 6   student                     5000 non-null   object
 7   lead_source                 5000 non-null   object
 8   contact_channel             5000 non-null   object
 9   site_visits_30d             5000 non-null   int64
 10  emails_opened_30d           5000 non-null   int64
 11  sms_clicked_30d             5000 non-null   int64
 12  app_installed               5000 non-null   object
 13  trial_class_attended        5000 non-null   object
 14  tours_completed             5000 non-null   object
 15  past_classes_90d            5000 non-null   int64
 16  time_since_last_visit_days  5000 non-null   int64
 17  prior_member                5000 non-null   object
 18  membership_status           5000 non-null   object
 19  price_sensitivity           5000 non-null   object
 20  fitness_goal                5000 non-null   object
 21  preferred_time              5000 non-null   object
 22  promo_type                  5000 non-null   object
 23  discount_pct                5000 non-null   int64
 24  satisfaction_score          5000 non-null   float64
 25  competitor_nearby           5000 non-null   object
 26  monthly_spend_usd           5000 non-null   float64
 27  accept_promo                5000 non-null   int64
dtypes: float64(4), int64(9), object(15)
memory usage: 1.1+ MB
```

```python
# Target distribution
plt.figure(figsize=(8, 6))
sns.histplot(data=df, x="accept_promo", bins=2)
plt.title("Distribution of Promo Acceptance (accept_promo)")
plt.xlabel("accept_promo (0=No, 1=Yes)")
plt.ylabel("Count")
plt.xticks([0, 1])
plt.show()

# Display counts
accept_counts = df["accept_promo"].value_counts()
accept_pct = df["accept_promo"].value_counts(normalize=True) * 100

print("Counts of accept_promo:")
print(accept_counts)
print("\nPercentages of accept_promo:")
print(accept_pct.round(2))
```



```
Counts of accept_promo:
accept_promo
0    4182
1     818
Name: count, dtype: int64

Percentages of accept_promo:
accept_promo
0    83.64
1    16.36
Name: proportion, dtype: float64
```

```python
# Data preparation (robust dummy encoding)
# Drop ID column
df_model = df.drop(columns=["customer_id"])
# Automatically identify categorical columns
cat_vars = df_model.select_dtypes(include=["object"]).columns.tolist()
# Remove target from the list if it appears as object (just in case)
if "accept_promo" in cat_vars:
    cat_vars.remove("accept_promo")
print("Categorical columns detected:", cat_vars)

# Create dummy variables for all categorical predictors
df_model = pd.get_dummies(df_model, columns=cat_vars, drop_first=True)

# Safety check: confirm no object columns remain (besides none)
print("\nObject columns remaining after encoding:",
      df_model.select_dtypes(include=["object"]).columns.tolist())

print("\nPrepared dataset shape:", df_model.shape)
print(df_model.head())
```

```
Categorical columns detected: ['gender', 'student', 'lead_source', 'contact_channel', 'app_installed', 'trial_class_attended', 'tours_completed', 'prior_
member', 'membership_status', 'price_sensitivity', 'fitness_goal', 'preferred_time', 'promo_type', 'competitor_nearby']

Object columns remaining after encoding: []

Prepared dataset shape: (5000, 39)
```

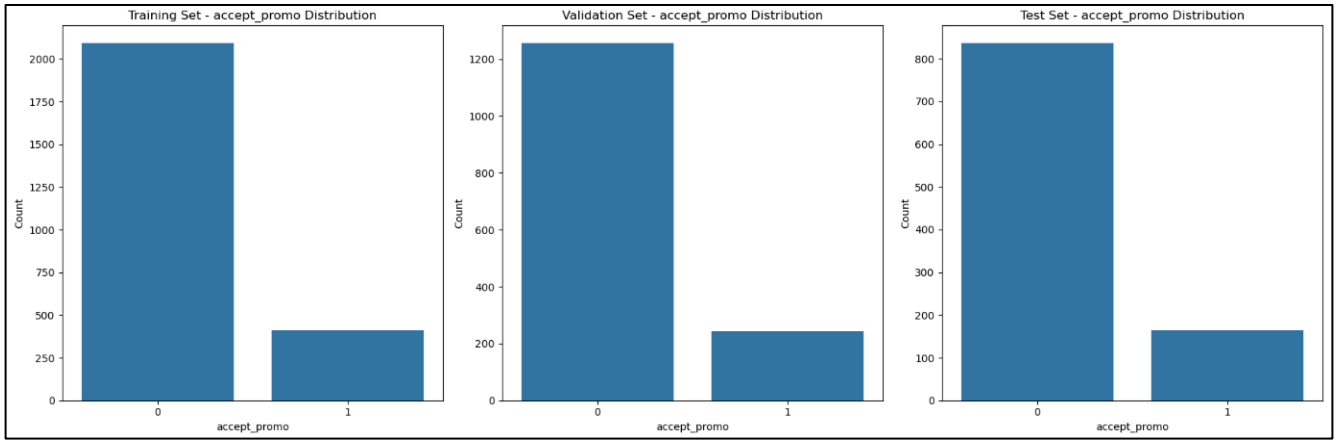
```
   age  income_k  distance_miles  commute_minutes  site_visits_30d  \
0   57      39.5             0.5                4                4
1   43     108.8             1.3               10                8
2   33      96.8             8.1               46                6
3   25      59.3             1.7                9                4
4   56      98.9             6.5               49                4

   emails_opened_30d  sms_clicked_30d  past_classes_90d  \
0                  2                1                 0
1                  2                1                 2
2                  1                2                 3
3                  1                2                 2
4                  7                0                10

   time_since_last_visit_days  discount_pct  ...  price_sensitivity_Low  \
0                          37             8  ...                   True
1                          28            12  ...                   True
2                         159            12  ...                  False
3                          13            10  ...                  False
4                          74             8  ...                  False

   price_sensitivity_Med  fitness_goal_Rehab  fitness_goal_Strength  \
0                  False               False                  False
1                  False               False                  False
2                   True               False                  False
3                   True               False                  False
4                  False               False                  False

   fitness_goal_Weight Loss  preferred_time_Evening  preferred_time_Morning  \
0                     False                   False                   False
1                      True                    True                   False
2                      True                    True                   False
3                     False                   False                    True
4                     False                    True                   False

   promo_type_Free Trial  promo_type_Percent Off  competitor_nearby_Yes
0                  False                   False                  False
1                   True                   False                  False
2                   True                   False                   True
3                  False                    True                  False
4                  False                   False                  False

[5 rows x 39 columns]
```

```python
# Block 5: Define target + predictors, then Train/Val/Test split (50/30/20)

target = "accept_promo"

# Features = everything except the target
features = df_model.drop(columns=[target])
target_variable = df_model[target]

# First split out TEST (20%)
X_train, X_test, y_train, y_test = train_test_split(
    features, target_variable,
    test_size=0.20,
    random_state=42,
    stratify=target_variable
)

# Split remaining TRAIN (80%) into Train (50%) and Validation (30%)
# 0.375 * 0.80 = 0.30 overall
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train,
    test_size=0.375,
    random_state=42,
    stratify=y_train
)

print("Training set shape:", X_train.shape, y_train.shape)
print("Validation set shape:", X_val.shape, y_val.shape)
print("Test set shape:", X_test.shape, y_test.shape)
print("\nAcceptance rate (Train):", round(y_train.mean(), 3))
print("Acceptance rate (Val):  ", round(y_val.mean(), 3))
print("Acceptance rate (Test): ", round(y_test.mean(), 3))
```

```
Training set shape: (2500, 38) (2500,)
Validation set shape: (1500, 38) (1500,)
Test set shape: (1000, 38) (1000,)

Acceptance rate (Train): 0.164
Acceptance rate (Val):   0.163
Acceptance rate (Test):  0.164
```

```python
# Plot target distribution across Train/Val/Test
train_data = X_train.copy()
train_data["accept_promo"] = y_train

val_data = X_val.copy()
val_data["accept_promo"] = y_val

test_data = X_test.copy()
test_data["accept_promo"] = y_test

fig, axs = plt.subplots(1, 3, figsize=(18, 6))

sns.countplot(data=train_data, x="accept_promo", ax=axs[0])
axs[0].set_title("Training Set - accept_promo Distribution")
axs[0].set_xlabel("accept_promo")
axs[0].set_ylabel("Count")
axs[0].set_xticks([0, 1])

sns.countplot(data=val_data, x="accept_promo", ax=axs[1])
axs[1].set_title("Validation Set - accept_promo Distribution")
axs[1].set_xlabel("accept_promo")
axs[1].set_ylabel("Count")
axs[1].set_xticks([0, 1])

sns.countplot(data=test_data, x="accept_promo", ax=axs[2])
axs[2].set_title("Test Set - accept_promo Distribution")
axs[2].set_xlabel("accept_promo")
axs[2].set_ylabel("Count")
axs[2].set_xticks([0, 1])
```

```python
sns.countplot(data=test_data, x="accept_promo", ax=axs[2])
axs[2].set_title("Test Set - accept_promo Distribution")
axs[2].set_xlabel("accept_promo")
axs[2].set_ylabel("Count")
axs[2].set_xticks([0, 1])

plt.tight_layout()
plt.show()

# Print counts and percentages
def counts_and_pct(series):
    counts = series.value_counts()
    pct = series.value_counts(normalize=True) * 100
    return counts, pct

train_counts, train_pct = counts_and_pct(train_data["accept_promo"])
val_counts, val_pct = counts_and_pct(val_data["accept_promo"])
test_counts, test_pct = counts_and_pct(test_data["accept_promo"])

print("Training set accept_promo counts and percentages:")
print(train_counts)
print(train_pct.round(2))

print("\nValidation set accept_promo counts and percentages:")
print(val_counts)
print(val_pct.round(2))

print("\nTest set accept_promo counts and percentages:")
print(test_counts)
print(test_pct.round(2))
```




```
Training set accept_promo counts and percentages:
accept_promo
0    2091
1     409
Name: count, dtype: int64
accept_promo
0    83.64
1    16.36
Name: proportion, dtype: float64

Validation set accept_promo counts and percentages:
accept_promo
0    1255
1     245
Name: count, dtype: int64
accept_promo
0    83.67
1    16.33
Name: proportion, dtype: float64

Test set accept_promo counts and percentages:
accept_promo
0    836
1    164
Name: count, dtype: int64
accept_promo
0    83.6
1    16.4
Name: proportion, dtype: float64
```

```python
# Train Logistic Regression model (baseline)
logistic_model = LogisticRegression(random_state=42, max_iter=2000)
logistic_model.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")
```

```
Logistic Regression model trained successfully.
```

```python
# Predictions (classes) + predicted probabilities

# Predicted classes
y_train_pred = logistic_model.predict(X_train)
y_val_pred   = logistic_model.predict(X_val)
y_test_pred  = logistic_model.predict(X_test)

# Predicted probabilities for class 1 (accept_promo = 1)
y_train_proba = logistic_model.predict_proba(X_train)[:, 1]
y_val_proba   = logistic_model.predict_proba(X_val)[:, 1]
y_test_proba  = logistic_model.predict_proba(X_test)[:, 1]

print("Predictions (classes and probabilities) made successfully.")
print("\nSample predicted probabilities (first 10, Test):")
print(pd.Series(y_test_proba).head(10).round(4))
```

```
Predictions (classes and probabilities) made successfully.

Sample predicted probabilities (first 10, Test):
0    0.0942
1    0.0318
2    0.6745
3    0.0450
4    0.0581
5    0.4625
6    0.0854
7    0.3566
8    0.0449
9    0.0971
dtype: float64
```

```python
# Coefficients + Logistic Regression Formula
# Extract coefficients and intercept

feature_names = X_train.columns
coefficients = logistic_model.coef_[0]
intercept = logistic_model.intercept_[0]

# Create coefficient table
coef_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients
})

# Add intercept
intercept_df = pd.DataFrame({
    "Feature": ["Intercept"],
    "Coefficient": [intercept]
})

coef_df = pd.concat([intercept_df, coef_df], ignore_index=True)

print("Logistic Regression Coefficients (including Intercept):")
print(coef_df)

# Logistic Regression Formula

linear_combination = f"{intercept:.4f}"
for i, feature in enumerate(feature_names):
    linear_combination += f" + {coefficients[i]:.4f}*{feature}"

formula = f"p = 1 / (1 + exp(-({linear_combination})))"

print("\nLogistic Regression Formula (probability of accept_promo = 1):")
print(formula)
```

Logistic Regression Coefficients (including Intercept):

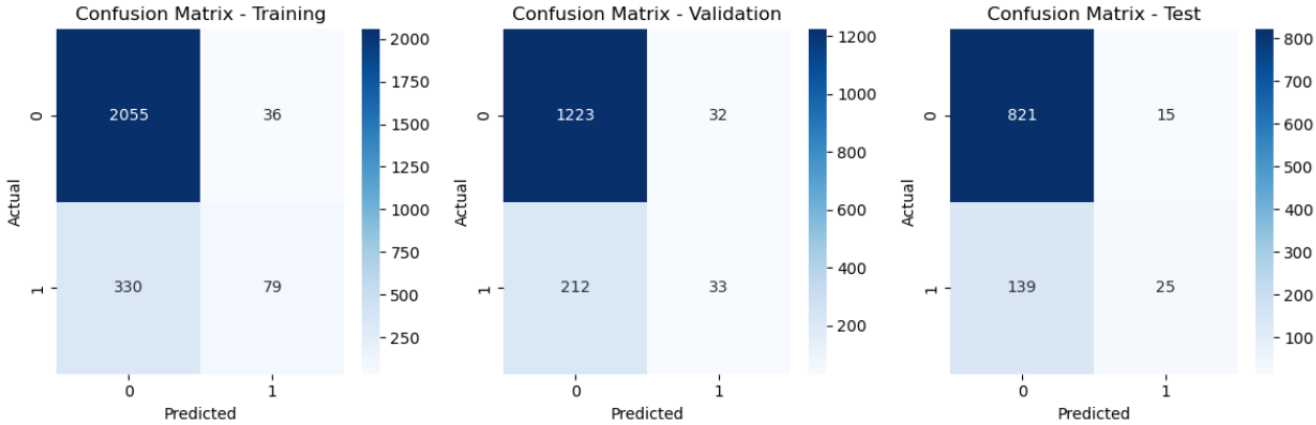
	Feature	Coefficient
0	Intercept	-3.582312
1	age	0.003036
2	income_k	0.009359
3	distance_miles	0.004675
4	commute_minutes	-0.022529
5	site_visits_30d	-0.018427
6	emails_opened_30d	0.093035
7	sms_clicked_30d	0.164597
8	past_classes_90d	0.091878
9	time_since_last_visit_days	-0.023522
10	discount_pct	0.027167
11	satisfaction_score	0.111062
12	monthly_spend_usd	-0.000755
13	gender_Male	-0.260534
14	student_Yes	0.006432
15	lead_source_Google Ads	0.081593
16	lead_source_Instagram	0.001818
17	lead_source_Other	0.187346
18	lead_source_Referral	0.619180
19	lead_source_Walk-in	0.397699
20	contact_channel_Phone	-0.346110
21	contact_channel_SMS	0.190314
22	app_installed_Yes	0.095240
23	trial_class_attended_Yes	0.931905
24	tours_completed_Yes	0.760164
25	prior_member_Yes	0.227027
26	membership_status_Cancelled	0.569537
27	membership_status_Frozen	-0.432610
28	membership_status_Never Member	-0.211225
29	price_sensitivity_Low	-0.099749
30	price_sensitivity_Med	-0.384031
31	fitness_goal_Rehab	-0.043077
32	fitness_goal_Strength	-0.037560
33	fitness_goal_Weight Loss	-0.195149
34	preferred_time_Evening	0.169485
35	preferred_time_Morning	0.009926
36	promo_type_Free Trial	0.171340
37	promo_type_Percent Off	0.195515
38	competitor_nearby_Yes	-0.266848

Logistic Regression Formula (probability of accept_promo = 1):

```
p = 1 / (1 + exp(-(-3.5823 + 0.0030*age + 0.0094*income_k + 0.0047*distance_miles + -0.0225*commute_minutes + -0.0184*site_visits_30d + 0.0930*emails_ope
ned_30d + 0.1646*sms_clicked_30d + 0.0919*past_classes_90d + -0.0235*time_since_last_visit_days + 0.0272*discount_pct + 0.1111*satisfaction_score + -0.00
08*monthly_spend_usd + -0.2605*gender_Male + 0.0064*student_Yes + 0.0816*lead_source_Google Ads + 0.0018*lead_source_Instagram + 0.1873*lead_source_Other
+ 0.6192*lead_source_Referral + 0.3977*lead_source_Walk-in + -0.3461*contact_channel_Phone + 0.1903*contact_channel_SMS + 0.0952*app_installed_Yes + 0.93
19*trial_class_attended_Yes + 0.7602*tours_completed_Yes + 0.2270*prior_member_Yes + 0.5695*membership_status_Cancelled + -0.4326*membership_status_Froze
n + -0.2112*membership_status_Never Member + -0.0997*price_sensitivity_Low + -0.3840*price_sensitivity_Med + -0.0431*fitness_goal_Rehab + -0.0376*fitness
_goal_Strength + -0.1951*fitness_goal_Weight Loss + 0.1695*preferred_time_Evening + 0.0099*preferred_time_Morning + 0.1713*promo_type_Free Trial + 0.1955
*promo_type_Percent Off + -0.2668*competitor_nearby_Yes)))
```

```python
# Confusion matrices (Train / Validation / Test)

cm_train = confusion_matrix(y_train, y_train_pred)
cm_val   = confusion_matrix(y_val, y_val_pred)
cm_test  = confusion_matrix(y_test, y_test_pred)

print("Confusion Matrix - Training Set:")
print(cm_train)

print("\nConfusion Matrix - Validation Set:")
print(cm_val)

print("\nConfusion Matrix - Test Set:")
print(cm_test)

# Visualize confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

sns.heatmap(cm_train, annot=True, fmt="d", cmap="Blues", ax=axes[0])
axes[0].set_title("Confusion Matrix - Training")
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("Actual")

sns.heatmap(cm_val, annot=True, fmt="d", cmap="Blues", ax=axes[1])
axes[1].set_title("Confusion Matrix - Validation")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("Actual")

sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues", ax=axes[2])
axes[2].set_title("Confusion Matrix - Test")
axes[2].set_xlabel("Predicted")
axes[2].set_ylabel("Actual")

plt.tight_layout()
plt.show()
```

```
Confusion Matrix - Training Set:
[[2055   36]
 [ 330   79]]

Confusion Matrix - Validation Set:
[[1223   32]
 [ 212   33]]

Confusion Matrix - Test Set:
[[821  15]
 [139  25]]
```



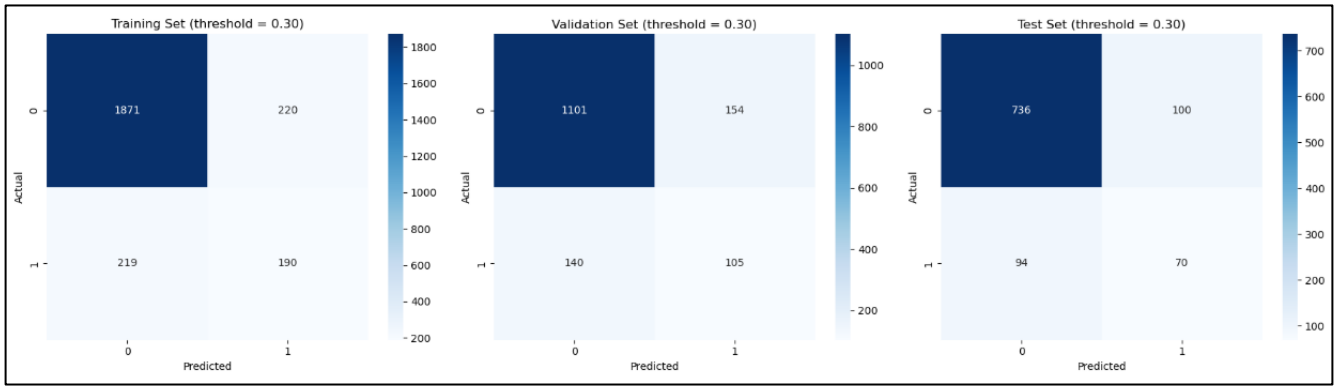

```python
# Classification reports

print("Classification Report - Training Set:")
print(classification_report(y_train, y_train_pred))

print("\nClassification Report - Validation Set:")
print(classification_report(y_val, y_val_pred))

print("\nClassification Report - Test Set:")
print(classification_report(y_test, y_test_pred))
```

```
Classification Report - Training Set:
              precision    recall  f1-score   support

           0       0.86      0.98      0.92      2091
           1       0.69      0.19      0.30       409

    accuracy                           0.85      2500
   macro avg       0.77      0.59      0.61      2500
weighted avg       0.83      0.85      0.82      2500


Classification Report - Validation Set:
              precision    recall  f1-score   support

           0       0.85      0.97      0.91      1255
           1       0.51      0.13      0.21       245

    accuracy                           0.84      1500
   macro avg       0.68      0.55      0.56      1500
weighted avg       0.80      0.84      0.80      1500


Classification Report - Test Set:
              precision    recall  f1-score   support

           0       0.86      0.98      0.91       836
           1       0.62      0.15      0.25       164

    accuracy                           0.85      1000
   macro avg       0.74      0.57      0.58      1000
weighted avg       0.82      0.85      0.80      1000
```

```python
# Change classification threshold to 0.30
# Set new threshold
threshold = 0.30

# Apply threshold to probabilities
y_train_pred_30 = (y_train_proba >= threshold).astype(int)
y_val_pred_30 = (y_val_proba >= threshold).astype(int)
y_test_pred_30 = (y_test_proba >= threshold).astype(int)

print(f"New classification threshold set to {threshold}")
```

```
New classification threshold set to 0.3
```

```python
# Confusion Matrices + Heatmaps (threshold = 0.30)

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

threshold = 0.30

# Predictions using threshold = 0.30
y_train_pred_30 = (y_train_proba >= threshold).astype(int)
y_val_pred_30   = (y_val_proba   >= threshold).astype(int)
y_test_pred_30  = (y_test_proba  >= threshold).astype(int)
# Confusion matrices
cm_train_30 = confusion_matrix(y_train, y_train_pred_30)
cm_val_30   = confusion_matrix(y_val, y_val_pred_30)
cm_test_30  = confusion_matrix(y_test, y_test_pred_30)

# Plot heatmaps
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

sns.heatmap(cm_train_30, annot=True, fmt="d", cmap="Blues", ax=axes[0])
axes[0].set_title("Training Set (threshold = 0.30)")
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("Actual")
sns.heatmap(cm_val_30, annot=True, fmt="d", cmap="Blues", ax=axes[1])
axes[1].set_title("Validation Set (threshold = 0.30)")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("Actual")
sns.heatmap(cm_test_30, annot=True, fmt="d", cmap="Blues", ax=axes[2])
axes[2].set_title("Test Set (threshold = 0.30)")
axes[2].set_xlabel("Predicted")
axes[2].set_ylabel("Actual")

plt.tight_layout()
plt.show()
```



```python
# Variable Selection using Statistical Significance (p-values)
# Review p-values to identify significant predictors

import statsmodels.api as sm

# Make sure all columns are numeric (booleans -> 0/1) for statsmodels
X_train_sm = X_train.copy()
X_val_sm   = X_val.copy()
X_test_sm  = X_test.copy()

for col in X_train_sm.columns:
    if X_train_sm[col].dtype == bool:
        X_train_sm[col] = X_train_sm[col].astype(int)
        X_val_sm[col]   = X_val_sm[col].astype(int)
        X_test_sm[col]  = X_test_sm[col].astype(int)

# Add intercept term (constant)
X_train_sm = sm.add_constant(X_train_sm)

# Fit logistic regression using statsmodels
logit_model = sm.Logit(y_train, X_train_sm)
result = logit_model.fit(disp=False)

# Show full summary (includes p-values)
print(result.summary())
```

```
                           Logit Regression Results
==============================================================================
Dep. Variable:           accept_promo   No. Observations:                 2500
Model:                          Logit   Df Residuals:                     2462
Method:                           MLE   Df Model:                           37
Date:                Mon, 12 Jan 2026   Pseudo R-squ.:                  0.1878
Time:                        14:04:05   Log-Likelihood:                -904.81
converged:                       True   LL-Null:                       -1114.0
Covariance Type:            nonrobust   LLR p-value:                 4.254e-66
```

```
==============================================================================================
                                   coef    std err          z      P>|z|      [0.025      0.975]
----------------------------------------------------------------------------------------------
const                           -2.4127   5.02e+06  -4.81e-07      1.000   -9.84e+06    9.84e+06
age                              0.0030      0.004      0.683      0.495      -0.006       0.012
income_k                         0.0094      0.002      4.048      0.000       0.005       0.014
distance_miles                   0.0043      0.048      0.089      0.929      -0.091       0.099
commute_minutes                 -0.0225      0.010     -2.310      0.021      -0.042      -0.003
site_visits_30d                 -0.0188      0.031     -0.614      0.539      -0.079       0.041
emails_opened_30d                0.0942      0.042      2.243      0.025       0.012       0.176
sms_clicked_30d                  0.1666      0.053      3.129      0.002       0.062       0.271
past_classes_90d                 0.0921      0.025      3.723      0.000       0.044       0.141
time_since_last_visit_days      -0.0236      0.002     -9.847      0.000      -0.028      -0.019
discount_pct                     0.0269      0.010      2.565      0.010       0.006       0.047
satisfaction_score               0.1113      0.045      2.497      0.013       0.024       0.199
monthly_spend_usd               -0.0008      0.003     -0.316      0.752      -0.006       0.004
gender_Male                     -0.2665      0.122     -2.191      0.028      -0.505      -0.028
student_Yes                      0.0063      0.155      0.041      0.967      -0.297       0.309
lead_source_Google Ads           0.1248      0.219      0.569      0.569      -0.305       0.555
lead_source_Instagram            0.0441      0.225      0.196      0.845      -0.397       0.485
lead_source_Other                0.2367      0.262      0.904      0.366      -0.276       0.750
lead_source_Referral             0.6736      0.225      2.996      0.003       0.233       1.114
lead_source_Walk-in              0.4553      0.251      1.810      0.070      -0.038       0.948
contact_channel_Phone           -0.3664      0.255     -1.436      0.151      -0.866       0.134
contact_channel_SMS              0.1913      0.125      1.526      0.127      -0.054       0.437
app_installed_Yes                0.0960      0.123      0.783      0.434      -0.144       0.336
trial_class_attended_Yes         0.9487      0.126      7.502      0.000       0.701       1.197
tours_completed_Yes              0.7798      0.144      5.398      0.000       0.497       1.063
prior_member_Yes                -0.9837   5.02e+06  -1.96e-07      1.000   -9.84e+06    9.84e+06
membership_status_Cancelled      0.5801      0.229      2.530      0.011       0.131       1.030
membership_status_Frozen        -0.4733      0.368     -1.287      0.198      -1.194       0.247
membership_status_Never Member  -1.4290   5.02e+06  -2.85e-07      1.000   -9.84e+06    9.84e+06
price_sensitivity_Low           -0.1071      0.156     -0.688      0.491      -0.412       0.198
price_sensitivity_Med           -0.3971      0.145     -2.739      0.006      -0.681      -0.113
fitness_goal_Rehab              -0.0519      0.261     -0.199      0.843      -0.564       0.460
fitness_goal_Strength           -0.0455      0.157     -0.290      0.771      -0.353       0.262
fitness_goal_Weight Loss        -0.2048      0.148     -1.386      0.166      -0.494       0.085
preferred_time_Evening           0.1787      0.172      1.036      0.300      -0.159       0.517
preferred_time_Morning           0.0166      0.188      0.088      0.929      -0.352       0.385
promo_type_Free Trial            0.1809      0.182      0.994      0.320      -0.176       0.537
promo_type_Percent Off           0.2058      0.214      0.961      0.336      -0.214       0.625
competitor_nearby_Yes           -0.2727      0.126     -2.161      0.031      -0.520      -0.025
==============================================================================================
```

```python
# Create a clean p-value table (sorted)

pvals = result.pvalues.sort_values()
coefs = result.params

pval_table = pd.DataFrame({
    "feature": pvals.index,
    "coef": coefs[pvals.index].values,
    "p_value": pvals.values
})

print("P-values (sorted):")
print(pval_table)
```

```
P-values (sorted):
                          feature      coef       p_value
0       time_since_last_visit_days -0.023584  7.064708e-23
1         trial_class_attended_Yes  0.948717  6.283668e-14
2              tours_completed_Yes  0.779823  6.721659e-08
3                         income_k  0.009365  5.158461e-05
4                 past_classes_90d  0.092070  1.966153e-04
5                  sms_clicked_30d  0.166634  1.755152e-03
6             lead_source_Referral  0.673625  2.737955e-03
7            price_sensitivity_Med -0.397135  6.164331e-03
8                     discount_pct  0.026922  1.031460e-02
9      membership_status_Cancelled  0.580095  1.141242e-02
10              satisfaction_score  0.111329  1.250944e-02
11                 commute_minutes -0.022546  2.089307e-02
12               emails_opened_30d  0.094152  2.488601e-02
13                     gender_Male -0.266474  2.844373e-02
14           competitor_nearby_Yes -0.272653  3.071259e-02
15             lead_source_Walk-in  0.455258  7.023562e-02
16             contact_channel_SMS  0.191296  1.271025e-01
17           contact_channel_Phone -0.366380  1.510367e-01
18        fitness_goal_Weight Loss -0.204823  1.657555e-01
19        membership_status_Frozen -0.473349  1.979940e-01
20          preferred_time_Evening  0.178665  3.000717e-01
21           promo_type_Free Trial  0.180852  3.202563e-01
22          promo_type_Percent Off  0.205799  3.364455e-01
23               lead_source_Other  0.236679  3.658547e-01
24               app_installed_Yes  0.096016  4.336781e-01
25           price_sensitivity_Low -0.107072  4.913733e-01
26                             age  0.003017  4.946365e-01
27                  site_visits_30d -0.018822  5.394519e-01
28          lead_source_Google Ads  0.124816  5.691771e-01
29               monthly_spend_usd -0.000841  7.518890e-01
30           fitness_goal_Strength -0.045517  7.714706e-01
31              fitness_goal_Rehab -0.051913  8.425212e-01
32           lead_source_Instagram  0.044054  8.448182e-01
33                   distance_miles  0.004301  9.292503e-01
34          preferred_time_Morning  0.016628  9.294925e-01
35                      student_Yes  0.006330  9.673509e-01
36                            const -2.412721  9.999996e-01
37  membership_status_Never Member -1.429025  9.999998e-01
38                 prior_member_Yes -0.983696  9.999998e-01
```

```python
# Select significant variables (example: p < 0.05)

alpha = 0.05

significant_features = pval_table.loc[pval_table["p_value"] < alpha, "feature"].tolist()

# Ensure const stays in the model
if "const" not in significant_features:
    significant_features = ["const"] + significant_features

print("Significant features (p < 0.05):")
print(significant_features)
print("\nNumber of significant features:", len(significant_features) - 1)
```

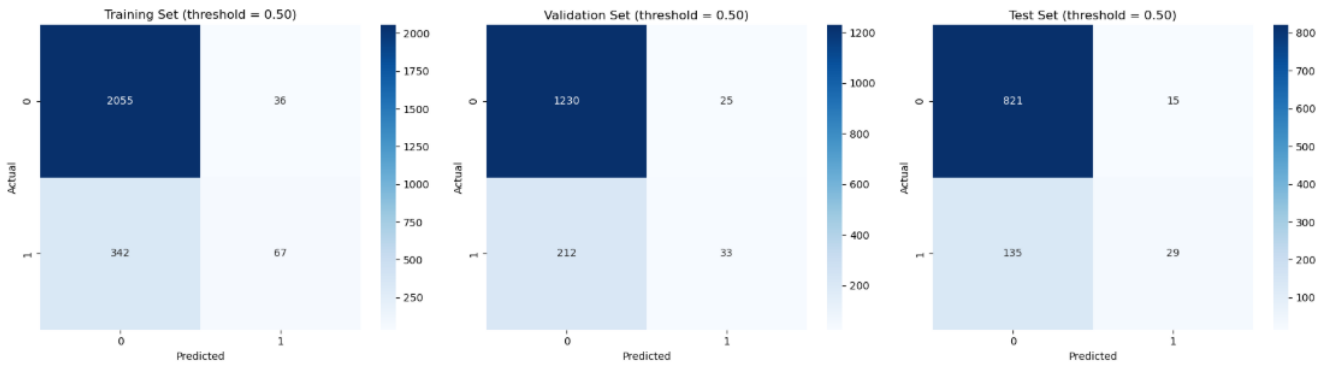
```
Significant features (p < 0.05):
['const', 'time_since_last_visit_days', 'trial_class_attended_Yes', 'tours_completed_Yes', 'income_k', 'past_classes_90d', 'sms_clicked_30d', 'lead_sour
ce_Referral', 'price_sensitivity_Med', 'discount_pct', 'membership_status_Cancelled', 'satisfaction_score', 'commute_minutes', 'emails_opened_30d', 'gen
der_Male', 'competitor_nearby_Yes']

Number of significant features: 15
```

```
Number of significant features: 15
```

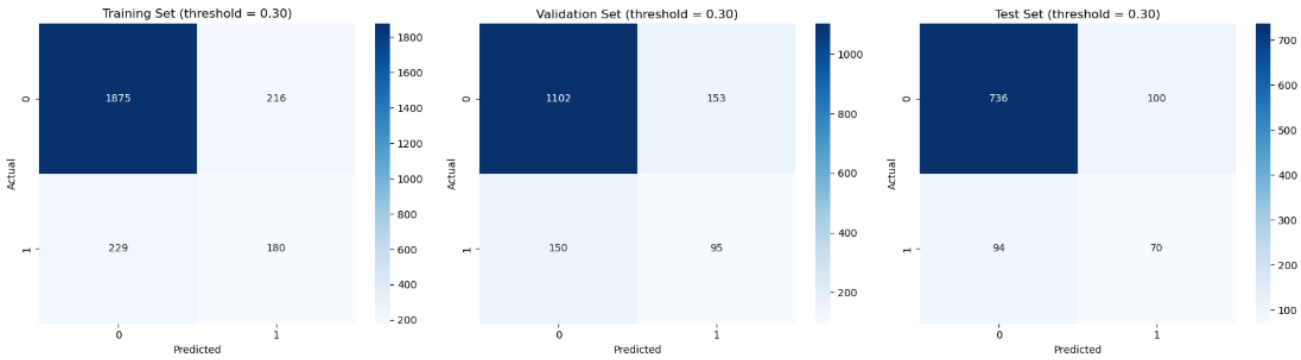
```python
# Refit simplified model using selected features
# Evaluate performance again

reduced_features = [f for f in significant_features if f != "const"]

X_train_red = X_train[reduced_features].copy()
X_val_red   = X_val[reduced_features].copy()
X_test_red  = X_test[reduced_features].copy()

# Train sklearn logistic regression on reduced feature set
logistic_model_red = LogisticRegression(random_state=42, max_iter=2000)
logistic_model_red.fit(X_train_red, y_train)

print("Reduced Logistic Regression model trained successfully.")
print("Number of features used:", X_train_red.shape[1])
```

```
Reduced Logistic Regression model trained successfully.
Number of features used: 15
```

```
=== Reduced Model (threshold = 0.50) ===

Confusion Matrix - Test:
[[821  15]
 [135  29]]

Classification Report - Test:
              precision    recall  f1-score   support

           0       0.86      0.98      0.92       836
           1       0.66      0.18      0.28       164

    accuracy                           0.85      1000
   macro avg       0.76      0.58      0.60      1000
weighted avg       0.83      0.85      0.81      1000
```



```
=== Reduced Model (threshold = 0.30) ===

Confusion Matrix - Test:
[[736 100]
 [ 94  70]]

Classification Report - Test:
              precision    recall  f1-score   support

           0       0.89      0.88      0.88       836
           1       0.41      0.43      0.42       164

    accuracy                           0.81      1000
   macro avg       0.65      0.65      0.65      1000
weighted avg       0.81      0.81      0.81      1000
```




```python
# Logistic Regression Formula (Reduced Model)
# Using only statistically significant variables
# Extracting coefficients and intercept from reduced model

reduced_feature_names = X_train_red.columns
reduced_coefficients = logistic_model_red.coef_[0]
reduced_intercept = logistic_model_red.intercept_[0]

# Create coefficient table
coef_red_df = pd.DataFrame({
    "Feature": reduced_feature_names,
    "Coefficient": reduced_coefficients
})

print("Reduced Logistic Regression Coefficients:")
print(coef_red_df)

print("\nIntercept:", reduced_intercept)

# Build logistic regression probability formula

linear_term = f"{reduced_intercept:.4f}"

for feature, coef in zip(reduced_feature_names, reduced_coefficients):
    sign = "+" if coef >= 0 else "-"
    linear_term += f" {sign} {abs(coef):.4f}*{feature}"

formula_reduced = f"p = 1 / (1 + exp(-({linear_term})))"

print("\nReduced Logistic Regression Formula (probability of accept_promo = 1):")
print(formula_reduced)
```

```
Reduced Logistic Regression Coefficients:
                        Feature  Coefficient
0    time_since_last_visit_days    -0.023458
1      trial_class_attended_Yes     0.943426
2           tours_completed_Yes     0.741440
3                      income_k     0.009429
4              past_classes_90d     0.094654
5               sms_clicked_30d     0.154896
6          lead_source_Referral     0.509454
7         price_sensitivity_Med    -0.290678
8                  discount_pct     0.032915
9   membership_status_Cancelled     0.915997
10           satisfaction_score     0.112466
11              commute_minutes    -0.021601
12            emails_opened_30d     0.085371
13                   gender_Male    -0.242198
14        competitor_nearby_Yes    -0.255168

Intercept: -3.493273671656133

Reduced Logistic Regression Formula (probability of accept_promo = 1):
p = 1 / (1 + exp(-(-3.4933 - 0.0235*time_since_last_visit_days + 0.9434*trial_class_attended_Yes + 0.7414*tours_completed_Yes + 0.0094*income_k + 0.0947
*past_classes_90d + 0.1549*sms_clicked_30d + 0.5095*lead_source_Referral - 0.2907*price_sensitivity_Med + 0.0329*discount_pct + 0.9160*membership_status
_Cancelled + 0.1125*satisfaction_score - 0.0216*commute_minutes + 0.0854*emails_opened_30d - 0.2422*gender_Male - 0.2552*competitor_nearby_Yes)))
```